

Lab overview

In this Amazon Web Services (AWS) lab, I'll gain hands-on experience in deploying a serverless application programming interface (API) by using the AWS Serverless Application Model (AWS SAM). I am guided through the process of understanding the purpose of each file in the Python-based source code, deploying a basic AWS Lambda function, and ultimately implementing a more advanced function to read the contents of an Amazon Simple Storage Service (Amazon S3) object.

Objectives

By the end of this lab, I'll have done the following:

- Accessed the preconfigured AWS Code Editor instance with the preloaded application code.
- Reviewed the three application files (`app.py`, `requirements.txt`, `template.yaml`) and their purposes.
- Built and deployed the application by using the `sam build` and `sam deploy` commands.
- Visit the API endpoint in a browser and use client URL (`curl`) commands to view the initial static text response.
- Modified the `app.py` file so that it uses the AWS SDK for Python (Boto3) to retrieve the contents from the `object1` file and include the file contents in the Lambda response.
- Redeploy the updated application and verify that it returns the contents of the `object1` file.

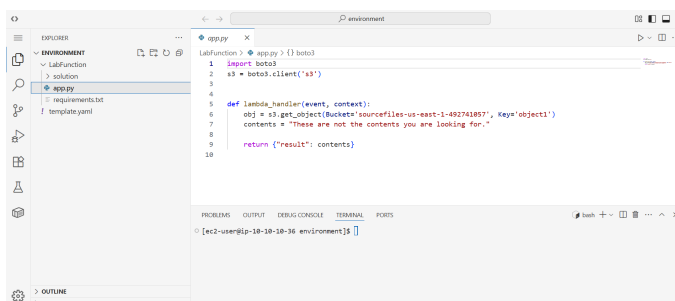
Task 1.1: Connect to AWS Code Editor

AWS Code Editor is a cloud-based integrated development environment (IDE) that one can use to write, run, and debug code with only a browser. It includes a code editor, debugger, and terminal. AWS Code Editor comes prepackaged with essential tools for popular programming languages, including JavaScript, Python, PHP, and more. One doesn't need to install files or configure a development machine to start new projects.

In this task, I'll connect to the AWS Code Editor environment that's provisioned as part of this lab.

Task 1.2: Review the application files

In this task, I'll review the application files, and learn about the main sections and what they do.



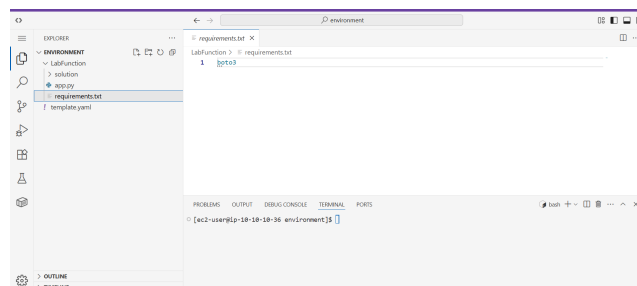
This Python code(**app.py**) is an AWS Lambda function that uses the SDK for Python library to interact with Amazon S3, a cloud storage service. My challenge is to see if I can spot the update that it needs to read the contents of an object.

- It imports the *boto3* library, which is the Amazon Web Services (AWS) SDK for Python.
- It creates an Amazon S3 client by using *boto3.client('s3')*, which is used to interact with the Amazon S3 service.
- It defines a function called *lambda_handler* that takes two arguments: *event* and *context*. This function is invoked when the Lambda function is run.
- Inside the *lambda_handler* function, it retrieves an object from the Amazon S3 bucket with the name that starts with *sourcefiles* and the key *object1*.
- It reads the data stored in the *contents* variable, which is only static text at this time.
- The function returns a dictionary that contains the key “result” and the data stored in the *contents* variable.

In summary, this Lambda function currently retrieves an object from an Amazon S3 bucket, and it also retrieves the contents of a specific variable. The function then returns the static text as a result.

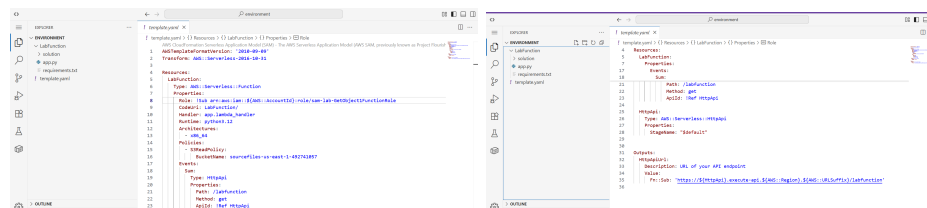
This scenario appears unusual to me. I am questioning the reason why a Lambda function fetches an object from an Amazon S3 bucket and returns the contents of a variable with static text, instead of the object1 file’s contents. I believe this is the section of the script that needs to be updated.

To review the requirements for the application, I'll open **requirements.txt**.



In this lab, the *requirements.txt* file contains *boto3* as a requirement because the lab uses SDK for Python *boto3* library to interact with AWS resources. *boto3* is the official AWS SDK for Python. I can use it to access and manage AWS services. By including *boto3* in the *requirements.txt* file, the *boto3* library is installed and available when I run the code in the lab. Because it's already installed, *boto3* library makes it possible for me to use AWS resources seamlessly within the Python code.

To review the resources that are created for this application, I'll open **template.yaml**.



This template is a YAML Markup Language (YAML) file that defines an AWS CloudFormation stack by using the AWS Serverless Application Model (AWS SAM) extension. The template creates a serverless application with the following resources:

AWSTemplateFormatVersion: This setting specifies the AWS CloudFormation template version that the template conforms to.

Transform: This directive instructs CloudFormation to apply the specified transform to the template. In this case, it's using the AWS SAM transform.

Resources:

- **LabFunction:** This is a Lambda function that's defined using the `AWS::Serverless::Function` resource type. The properties of this function are:
 - **CodeUri:** This specifies the location of the Lambda function's source code.
 - **Handler:** This specifies the entry point of the Lambda function.
 - **Runtime:** This specifies the runtime environment for the Lambda function (Python 3.9).
 - **Architectures:** This specifies the architecture for the Lambda function (x86_64).
 - **Policies:** This specifies the IAM policies (which are attached to the Lambda function), that allow the function to be read from the specified Amazon S3 bucket.
 - **Events:** Specifies the event sources for the Lambda function. In this case, it's an HTTP API event source with a GET method.
- **HttpApi:** This is an HTTP API that's defined using the `AWS::Serverless::HttpApi` resource type. The property **StageName** is set to `"$default"`.
- **Outputs:** This contains the output values that I can import into other stacks or return in the AWS Management Console.
 - **HttpApiUrl:** Provides the URL of the API endpoint, which is a combination of the HttpApi resource, AWS Region, and AWS URL suffix.

Task complete: I have successfully connected to the AWS Code Editor environment and familiarized myself with the interface. I reviewed the three application files that help make up the application to understand what they do and how they are used. You also identified the update that's required to read the contents of the `object1` file.

Task 2: Build and deploy the application

After examining the application code and learning about the functionality of the files in the development process, I am now ready to use the AWS Serverless Application Model (AWS SAM) to construct the application package and subsequently deploy the application within the AWS infrastructure. Both `sam build` and `sam deploy` are commands that I can use in the AWS SAM command line interface (CLI), a tool that helps me manage and deploy serverless applications on AWS. These commands perform specific tasks in the development and deployment processes of my serverless application.

sam build: The *sam build* command is used to compile and package my serverless application's source code and its dependencies. It processes my AWS SAM template, which defines the application's resources and configuration, and creates a build directory with the compiled artifacts (e.g., Lambda functions, layers, etc.) and transformed template.

The build process includes the following steps:

- Reading the AWS SAM template and identifying the resources to build.
- Building each resource (such as a Lambda function) by downloading the required dependencies, compiling the source code (if necessary), and creating a deployment package (such as a .zip file).
- Generating a new AWS SAM template with the updated resource references (for example, pointing to the built artifacts).

After running *sam build*, my serverless application is ready for deployment or local testing by using *sam local* commands.

sam deploy: The *sam deploy* command is used to deploy my serverless application to AWS. It takes the built artifacts and transformed template that's generated by *sam build*, and creates or updates the necessary AWS resources.

The deployment process includes the following steps:

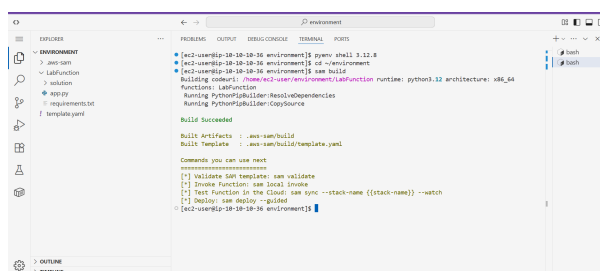
- Packaging the built artifacts and uploading them to an S3 bucket. This step can be skipped if the artifacts are already packaged and uploaded by using the *sam package* command.
- Deploying the application by using AWS CloudFormation. It creates a new CloudFormation stack, or updates an existing one with the resources that are defined in the transformed AWS SAM template. This step creates or updates AWS resources, such as Lambda functions, API Gateway endpoints, DynamoDB tables, and others.

During the deployment, I can specify various options, such as the stack name, deployment Region, parameter overrides, and more.

Task 2.1 Build the deployment package

The first step is to compile and build source code into a deployment package. To compile the deployment package, the terminal session needs to be in the root environment folder.

I'll build the deployment package by running the following command that gives the following expected result below:

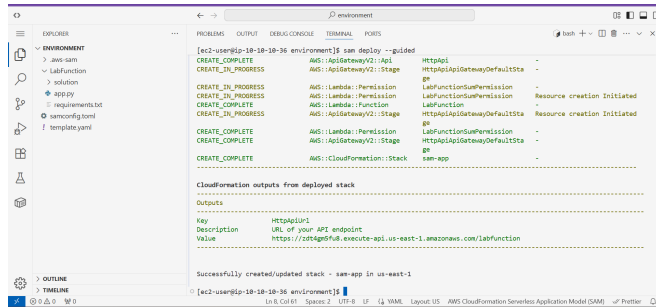


```
environment
[ec2-user@ip-10-10-10-10 environment]$ python shell 3.12.8
[ec2-user@ip-10-10-10-10 environment]$ cd ~
[ec2-user@ip-10-10-10-10 environment]$ sam build
Building console: /home/ec2-user/environment/LambdaFunction runtime: python3.12 architecture: x86_64
Functions: LambdaFunction
Running pythonpibuilder-readDependencies
Running pythonpibuilder-copySource
Build Succeeded

Build Artifacts : .aws-sam/build
Build Template : .aws-sam/build/template.yaml

Comments you can use next:
=====
[>] Validate SAM template: sam validate
[>] Invoke function: sam local invoke
[>] Test function in the Cloud: sam sync --stack-name {{stack-name}} --watch
[>] Deploy: sam deploy --guided
[ec2-user@ip-10-10-10-10 environment]$
```

I'll deploy the application with the following command, which will give the expected output result below:



```
[ec2-user@ip-10-10-10-36 environment]# sam deploy --guided
CREATE_COMPLETE      AWS::ApiGatewayV2::Api      HttpApi
CREATE_IN_PROGRESS   AWS::ApiGatewayV2::Stage     HttpApiStage
CREATE_IN_PROGRESS   AWS::Lambda::Permission      LambdaPermission
CREATE_IN_PROGRESS   AWS::Lambda::Permission      LambdaPermission
CREATE_IN_PROGRESS   AWS::Lambda::Function         LambdaFunction
CREATE_IN_PROGRESS   AWS::ApiGatewayV2::Stage     HttpApiStage
CREATE_COMPLETE      AWS::Lambda::Permission      LambdaPermission
CREATE_COMPLETE      AWS::ApiGatewayV2::Stage     HttpApiStage
CREATE_COMPLETE      AWS::CloudFormation::Stack    sam-app

CloudFormation outputs from deployed stack
Outputs
Key      HttpApiUrl
Description URL of your API endpoint
Value    https://zdt4gm5fu8.execute-api.us-east-1.amazonaws.com/labfunction

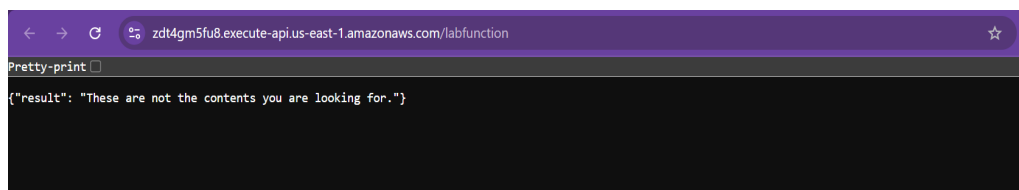
Successfully created/updated stack - sam-app in us-east-1
```

Task complete: I have successfully created the deployment package that encompasses my application's source code. I have also deployed the application into the AWS environment by using AWS SAM commands.

Task 3: Test the application

In this task, I'll visit the API endpoint in a browser tab to see that the application has been successfully deployed. I'll also test the application from a terminal, using a `curl` command. Using a `curl` command to test an AWS API Gateway endpoint allows me to send HTTP requests to the API Gateway and receive a response. This can help me verify if the API Gateway is functioning correctly, if the integration with backend services (like AWS Lambda) is working as expected, and if the API is returning the expected response data. Both methods of testing the application should return a response of *These are not the contents you are looking for.* if the deployment was completed successfully.

```
[ec2-user@ip-10-10-10-36 environment]$ echo;curl https://zdt4gm5fu8.execute-api.us-east-1.amazonaws.com/labfunction echo;echo
{"result": "These are not the contents you are looking for."}curl: (6) Could not resolve host: echo
[ec2-user@ip-10-10-10-36 environment]$
```

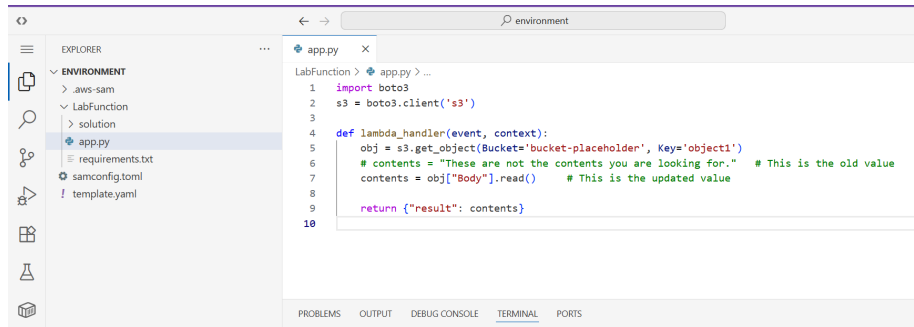


```
zdt4gm5fu8.execute-api.us-east-1.amazonaws.com/labfunction
Pretty-print
{"result": "These are not the contents you are looking for."}
```

Task complete: I have successfully tested the functionality of the application by using a browser and a `curl` command to access the API endpoint. Both test methods return the data that's stored in the *contents* variable, which is currently only static text.

Task 4: Update the application and test the changes

In this task, I'll update the application to read the contents of the object1 file's body. The application then populates the *contents* variable with this code snippet, instead of the static text that's currently being set in the *contents* variable.



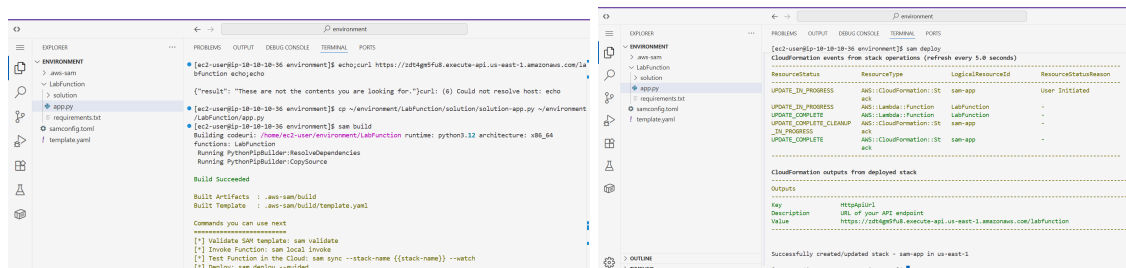
- `s3.get_object()` is a method from the SDK for Python that retrieves an object from an S3 bucket. The *Bucket* and *Key* parameters specify the bucket name and the object key, respectively. In this case, the bucket name starts with *sourcefiles* and the object key is *object1*.
- The `get_object()` method returns a dictionary that contains the object's metadata and its body as a *botocore.response.StreamingBody* object. The *obj* variable holds this dictionary.
- `obj["Body"]` accesses the *StreamingBody* object that's associated with the 'Body' key in the *obj* dictionary.
- The `read()` method is called on the *StreamingBody* object to read the contents of the object.

Task complete: I have successfully updated the app.py file to read the contents of the object1 file in the S3 bucket and update the *contents* variable with that code snippet. When the application runs, it replaces the value of the *contents* variable from the code snippet to the data from the object1 file.

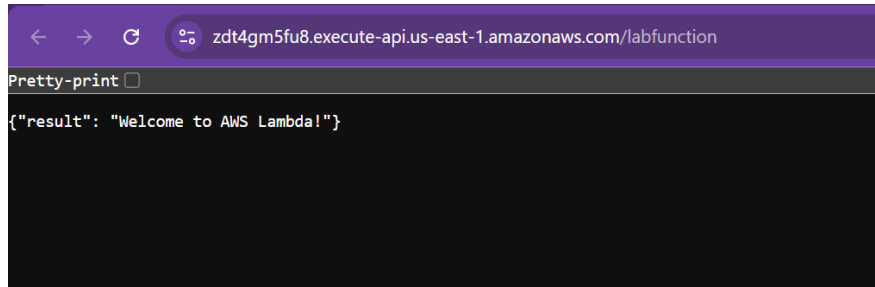
Task 4.1 - Build, deploy, and test the updated application

I have made the necessary changes to the application. I can now run the *sam build* and *sam deploy* commands to deploy the latest version of the application, and then test again.

To package the latest changes into a deployment file, I'll run the following commands, which will the following expected output below:



I'll test the functionality of the application using a web browser:

A screenshot of a web browser window. The address bar shows the URL 'zdt4gm5fu8.execute-api.us-east-1.amazonaws.com/labfunction'. Below the address bar, there is a 'Pretty-print' button. The main content area displays a JSON response:

```
{ "result": "Welcome to AWS Lambda!" }
```

Task complete: I have successfully updated the application to retrieve the contents of the *object1* file. I verified the updates by using a browser and a curl command to access the API endpoint. Both testing methods return the data that's stored in the *contents* variable, which is now the data from the *object1* file.

Conclusion

I now have successfully:

- Accessed the preconfigured AWS Code Editor instance with the preloaded application code.
- Reviewed the three application files (*app.py*, *requirements.txt*, *template.yaml*) and their purposes.
- Built and deployed the application by using the *sam build* and *sam deploy --guided* commands.
- Visited the API endpoint in a browser and used client URL (curl) commands to view the initial static text response.
- Modified the *app.py* file so that it uses the AWS SDK for Python (Boto3) to retrieve the contents from the *object1* file and include the file contents in the Lambda response.
- Redeployed the updated application and verified that it returns the contents of the *object1* file.